

The Benchmark Module

The measurement core: entities, ports, thirteen services, adapters and wiring

EdgePilot AI - Team 15
Documentation set, 23 August 2026
13 documents in this volume

src/modules/ - the domain modules

Work packages: benchmark, and vision benchmark.

What this folder is

Two modules, each with the same three layers, each able to change without the other:

- core/ - domain logic. Pure. Imports nothing from infrastructure/.
- application/ - use cases and the typed envelopes they return.
- infrastructure/ - adapters. HTTP clients, Prisma, the filesystem.

What happened here

modules/device/ was removed entirely. It held a self-reported hardware profile that no measurement ever consumed - the user typed how much RAM they had and nothing checked it. Once hardware fit became a VRAM residency reading from the runtime, the typed value had no consumer left and could only disagree with reality.

The benchmark module grew by eight core services and two infrastructure adapters. Every one of them replaced something that was previously a constant, an assumption, or nothing at all.

The two modules deliberately **duplicate** a small amount of transport code rather than sharing it - `http.ts` exists in both, with a comment saying the copy is intentional, because the two modules must be able to change independently. A shared helper here would couple the vision workload's release cycle to the benchmark layer's.

Subfolders

Module	What it does
benchmark/	Text and code generation benchmarks across local and cloud providers
vision-benchmark/	Image classification benchmarks

Connected folders

- src/app/api/v1/ - the routes that dispatch here.
- tests/ - mirrors both modules.
- docs/internal/ - `architecture.md`.

Referenced from

These folders point here. Each link below resolves in both directions.

- scripts/
- src/
- src/app/

src/modules/benchmark/ - the measurement core

Work package: AI / local model & DevOps - the benchmark and provider layer.

What this folder is

The part of EdgePilot that produces numbers, and the part that decides which numbers it is allowed to produce.

Three layers, and the direction of dependency is the point: `core/` imports nothing from `infrastructure/`. The ports are two interface files. The single place an adapter is constructed is `infrastructure/container.ts`. No API route in this project builds a provider, reads an environment variable, or touches Prisma - which is exactly what makes this module testable with a fake `fetch` and a stepped clock.

What happened here

This is where most of the fix list landed. In summary:

- `hardwareFit` stopped being the constant 50 and became a VRAM residency measurement read from the runtime.
- The first iteration of every run is now a **discarded cold start**. On the same 8B model, time-to-first-token was 36,445 ms cold and 369 ms warm. Averaging that in describes how recently someone used the model, not the model.
- Privacy left the readiness average and became an ordinal class that annotates or disqualifies a recommendation.
- Comparison gained a planner (local models must not run concurrently - they contend for the same GPU) and a report that refuses to declare a winner when the observed ranges overlap.
- Modality and task compatibility became real constraints instead of unread columns.
- A pre-flight egress warning answers "is this prompt about to leave the machine" **before** the request rather than after it.

Subfolders

Layer	What is in it
<code>core/</code>	Entities, ports, and 13 domain services
<code>application/</code>	DTOs, the runner, and the two use cases
<code>infrastructure/</code>	Provider adapters, Prisma repositories, config, the composition root

Connected folders

- `src/app/api/v1/benchmarks/, comparisons/, providers/, local-runtime/, readiness/[id]/ ( FOLDER.md)` - the HTTP surface.
- `tests/benchmark/` - 19 suites.
- `docs/benchmark/` - the branch guide and the research log.
- `scripts/benchmark/` - the evidence producers.

Referenced from

These folders point here. Each link below resolves in both directions.

- `docs/`

- evidence/benchmark/
- src/modules/

src/modules/benchmark/application/ - use cases and envelopes

Work package: benchmark and provider layer.

What this folder is

The layer between HTTP and the domain. It orchestrates: validate, measure, persist, return.

What happened here

The runner grew by 269 differing lines, carrying the cold-start iteration and the residency probe. The use case shrank slightly while gaining an ownership rule that had been implicit.

The design decision that survived every change: **persistence is layered on top of measurement, not inside it.** BenchmarkRunner knows nothing about the database, so when Postgres is unreachable mid-run the measurement is still returned to the caller with `persisted: false`, rather than being thrown away. A benchmark that ran is evidence whether or not it was saved.

Subfolders

Subfolder	What is in it
dtos/	The typed measurement envelope and the request schema
services/	BenchmarkRunner
use-cases/	RunBenchmark, RunComparison

Connected folders

- ../core/ - the domain services it calls.
- ../infrastructure/ - the adapters it is handed.
- tests/benchmark/.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/modules/benchmark/

src/modules/benchmark/core/ - the domain

Work package: benchmark and provider layer.

What this folder is

Pure domain logic. Nothing in here imports from `infrastructure/`, nothing here performs I/O, and nothing here knows what HTTP is. That constraint is what lets the whole scoring layer be tested without a network, a model, or a database.

What happened here

Eight new service files, 1,761 lines between them, and every one of them replaced something that was previously a constant, an assumption, or nothing at all. The single deletion that matters as much as the additions: `ReadinessCalculator` lost its privacy term.

Subfolders

Subfolder	What is in it
entities/	Benchmark, BenchmarkResult, ReadinessScore
ports/	The two interfaces the infrastructure layer implements
services/	10 files - scoring, classification, comparison, privacy

Connected folders

- `../application/` - what calls these.
- `../infrastructure/` - what implements the ports.
- `tests/benchmark/`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/`

src/modules/benchmark/infrastructure/ - adapters and wiring

Work package: benchmark and provider layer.

What this folder is

Everything that talks to something outside the process - HTTP, Prisma, the environment - plus the one place where it is all wired together.

What happened here

Two new adapters, and both of them exist to answer a question the application used to answer too late or not at all.

OllamaResidencyProbe.ts reads what actually happened to memory during a local run, which is what turned hardware fit from a constant into a measurement. OllamaCatalog.ts answers "is Ollama running and what does this machine have" **before** a benchmark is attempted rather than after one fails.

config.ts grew by 41 lines and container.ts by 37, both because there is more to wire.

Files

File	What it is
container.ts	78 lines. The composition root. Routes ask for a fully wired use case and get one; they never construct an adapter, read an environment variable, or touch Prisma. Every wiring decision is in this one readable place
config.ts	184 lines. Reads and validates the environment once, with documented defaults, so a misconfiguration surfaces as a named variable rather than a stack trace three layers down. Also refuses to evaluate in a browser bundle - every value here is a secret or a host that must stay server-side. The guard is a runtime check because server-only is not a dependency, and it is still worth having: it turns an accidental client import into an immediate, explicit failure at the point of the mistake
OllamaCatalog.ts	223 lines. Runtime status and the installed-model list. Exists because isConfigured() only reports whether a host string is set - whether anything is listening is a runtime question, and the user was learning the answer from a failed benchmark
OllamaResidencyProbe.ts	107 lines. GET /api/tags for size on disk, GET /api/ps for size (total resident, including the KV cache for the context window) and size_vram (how many of those bytes are on the GPU). size_vram / size is the fit signal, and it is reported by the runtime rather than inferred by us

Subfolders

Subfolder	What is in it
providers/	The five adapters, the registry, the error vocabulary, transport helpers
repositories/	The Prisma implementations

Connected folders

- ../core/ports/ - the interfaces implemented here.
- ../core/services/ - HardwareFitAssessor consumes the probe's observation.
- src/app/api/v1/ - every route calls container.ts and nothing else here.
- src/app/api/v1/local-runtime/ - the catalogue's endpoint.
- tests/benchmark/ - benchmark-config.test.ts, ollama-catalog.test.ts.

Referenced from

These folders point here. Each link below resolves in both directions.

- prisma/
- src/app/api/v1/benchmarks/
- src/modules/benchmark/
- src/modules/benchmark/application/
- src/modules/benchmark/application/services/
- src/modules/benchmark/core/

src/modules/benchmark/application/dtos/ - what goes in, what comes out

Work package: benchmark and provider layer.

What this folder is

Two files. `BenchmarkRequest.ts` validates what comes **in**. `BenchmarkMeasurement.ts` validates what goes **out**, and is the reason a reader can tell a measurement from a derivation from a placeholder.

What happened here

`BenchmarkMeasurement.ts` grew by 38 differing lines to carry the cold start as a first-class object rather than a footnote: `ColdStart` and `ColdStartSchema` exist so the discarded iteration has a name, a duration and a row of its own.

It is deliberately a **superset** of the scaffold's `BenchmarkResponseSchema` rather than a replacement. The dashboard was already being written against the existing response shape, so every field it had is still there, in the same place, with the same name, and everything new was added inside. That is why `src/shared/types/benchmark.ts` needed only three lines changed.

Files

File	What it is
<code>BenchmarkMeasurement.ts</code>	227 lines. <code>MeasuredIteration</code> , <code>MeasurementSummary</code> , <code>ColdStart</code> , <code>FallbackAttempt</code> , <code>BenchmarkRun</code> , <code>MeasurementStatus</code> , <code>ProviderErrorCode</code> , and <code>summarise()</code> - the function where a number would be invented if anywhere
<code>BenchmarkRequest.ts</code>	28 lines. <code>BenchmarkRequestSchema</code> and the scaffold's <code>BenchmarkResponseSchema</code>

Connected folders

- `../services/` - `BenchmarkRunner` produces these.
- `src/components/dashboard/` - `api.ts` imports these types with `import type`, so the dashboard cannot drift from the envelope without a compile error.
- `tests/benchmark/` - `benchmark-measurement.test.ts` is about what `summarise()` **refuses** to report.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/application/`
- `src/shared/`
- `src/shared/types/`

src/modules/benchmark/application/services/ - the runner

Work package: benchmark and provider layer.

What this folder is

One file, and the most-changed file in the module: 662 lines, 269 of them differing from the team's version.

It runs one benchmark request against the provider chain and turns raw adapter output into the typed envelope. It does no I/O of its own beyond the providers it is handed, and it knows nothing about the database.

What happened here

The cold start. The runner now runs N+1 iterations and discards the first. Measured on the same 8B model in one session: TTFT 36,445 ms cold against 369 ms warm, a factor of 99, with readiness swinging 75 to 90 on load state alone. The discarded iteration is not deleted - it is reported with its own duration and stored with `warmup: true`, so the arithmetic can be checked from the database rows.

The counting bug that followed. With the warm-up added, a request for three iterations reported "4/4 iterations succeeded". Success, dominant failure code and the summary now all read from `responses.slice(1)`. The fix went into the runner rather than into the assertion that caught it.

The residency probe. A pre-run probe records what memory looked like before the model was asked anything, which is what hardware fit is computed against. Its result is null-guarded explicitly - the first version's `catch` produced the right value for the wrong reason, which would have masked any other fault.

The fallback policy, unchanged and worth restating: fall back only when the provider failed for a reason another provider could plausibly not share.

Files

File	What it is
BenchmarkRunner.ts	663 lines. The run loop, the cold start, the residency probe, the fallback chain, and <code>DEFAULT_UNMEASURED_INPUTS</code>

Connected folders

- `../dtos/` - the envelope it produces.
- `../use-cases/` - `RunBenchmark`, which persists what it returns.
- `../../core/services/` - hardware fit, readiness, task compatibility.
- `../../infrastructure/` - `OllamaResidencyProbe.ts`.
- `prisma/migrations/` - the warmup column.
- `tests/benchmark/` - `benchmark-runner.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/modules/benchmark/application/

src/modules/benchmark/application/use-cases/ - orchestration

Work package: benchmark and provider layer.

What this folder is

Two use cases: run a benchmark and persist it, and compare several runs without persisting anything.

What happened here

`RunComparison.ts` is new. `RunBenchmark.ts` changed by 90 lines, and two of its decisions are worth reading before anyone changes them - both are written into the file's own header.

Where `userId` comes from. Ownership is derived from the workload the request names: `workloads.user_id` is required, and the benchmark is about that workload. This began as an interim rule while there was no session. It survived now that there *is* one, because it is still the right answer - a benchmark belongs to whoever owns the thing being benchmarked, and the workload row is already owned by the session that created it.

Why `RunComparison` does not persist. A comparison is an analysis of runs, not a new kind of record. Making it depend on a workload row would tie a read-only question to the write path. The individual outcomes come back in full, so a caller that wants to store them can.

`RunComparison` also honours the plan's execution mode rather than always reaching for `Promise.all` - which matters, because running two local models concurrently measures GPU contention rather than the models.

Files

File	What it is
<code>RunBenchmark.ts</code>	309 lines. Validate the referenced rows, run, persist, return. Includes <code>statusForFailedRun</code> and the graceful-degradation path when the database is down mid-run
<code>RunComparison.ts</code>	213 lines. Decide comparability, decide execution mode, run, report per-dimension verdicts

Connected folders

- `../services/` - the runner both call.
- `../../../../core/services/` - `ComparisonPlanner` and `ComparisonReport`.
- `../../../../infrastructure/repositories/` - persistence and the context lookup.
- `src/app/api/v1/benchmarks/` and `comparisons/` - the routes.
- `tests/benchmark/` - `run-benchmark.test.ts`, `run-comparison.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/application/`

src/modules/benchmark/core/entities/ - domain entities

Work package: benchmark and provider layer.

What this folder is

One file holding the three shapes the domain reasons about.

What happened here

Benchmark.ts changed by 14 lines: deviceId came off Benchmark, and ReadinessScore gained a nullable hardwareFit and a privacyClass. Those two type changes are the domain-level statement of two decisions - that a hardware fit may be genuinely unknown rather than defaulted, and that privacy is a class and not a score.

Files

File	What it is
Benchmark.ts	49 lines. Benchmark, BenchmarkResult, ReadinessScore

Connected folders

- ../services/ - what computes these.
- ../../infrastructure/repositories/ - what maps them to and from Postgres.
- prisma/ - the tables they correspond to.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/modules/benchmark/core/

src/modules/benchmark/core/ports/ - the boundary

Work package: benchmark and provider layer.

What this folder is

Two interface files, 44 lines together. They are the whole reason this module can be called hexagonal without qualification: the domain declares what it needs, and `infrastructure/` supplies it.

Because these are interfaces and nothing else, the test suite can hand the application layer a `FakeProvider` and an in-memory repository and exercise the real runner and the real use case with no network and no database.

What happened here

Unchanged. That is the interesting part - eight new services, a new use case, a removed module and four migrations all landed without needing to move the boundary. A boundary that survives that much change is one that was drawn in the right place.

Files

File	What it is
<code>AIPProvider.ts</code>	26 lines. What a provider must be able to do
<code>BenchmarkRepository.ts</code>	18 lines. What persistence must be able to do

Connected folders

- `../../infrastructure/providers/` - five implementations of `AIPProvider`.
- `../../infrastructure/repositories/` - the Prisma implementation of `BenchmarkRepository`.
- `tests/benchmark/` - `helpers.ts` implements both with test doubles.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/core/`
- `src/modules/benchmark/infrastructure/`

src/modules/benchmark/core/services/ - scoring and judgement

Work package: benchmark and provider layer.

What this folder is

Ten files, and collectively they are the answer to "what is this application allowed to claim". Eight of them are new.

What happened here

Each file below records, in its own header, the thing it was written to stop. The short version:

- A **constant** was being presented as a hardware measurement.
- A **weighted average** was mixing legal exposure with throughput.
- A **mean** was being reported as a comparison result.
- A **declared task type** was being written to the database and read by nothing.
- **Two local models** were about to be benchmarked concurrently on one GPU.
- A **privacy verdict** was arriving after the prompt had already left.

Files

File	What it is
HardwareFitAssessor.ts	229 lines. Replaces hardwareFit: 50, a constant every device on earth received. Scores placement - did the model fit in GPU memory, and if not how much of it did - and deliberately not speed, because latency is already a readiness term and a slow CPU run would otherwise be punished twice. Reports VRAM taken and the spilled share in words, and says so when the runtime does not report card capacity
ReadinessCalculator.ts	89 lines. Four measured performance terms. Privacy was the fifth and is gone: a coarse label mapped to 30/60/100 let a fast provider average away a data policy that should have disqualified it, and implied an arithmetic relationship between privacy levels nobody can defend
PrivacyAssessor.ts	314 lines. Privacy as an ordinal class , with disqualifiers, egress classification and observed transport. Annotates or rules out a recommendation instead of nudging a number. Fails closed on a null endpoint
privacy-catalogue.ts	97 lines. Provider data-policy facts by slug and billing tier. Every value starts null and unverified on purpose - these are terms-of-service facts that differ by tier and change without notice, and filling them in from memory is how an evidence-based tool starts publishing stale claims confidently
egress-warning.ts	133 lines. Answers <i>before</i> the run: is this prompt about to leave the machine, and to whom. Pure and synchronous so any surface can use it. Reports the prompt's character count and never carries the prompt text - an object that warns text is about to leave must not contain a copy of it
ComparisonPlanner.ts	141 lines. Decides whether entrants may be compared at all, and whether they may run concurrently. For two cloud providers, parallel is right. For two local models it destroys the measurement: they contend for the same GPU and the runtime may evict one for the other
ComparisonReport.ts	506 lines. Per-dimension verdicts, each marked established or not. A mean is not a fact: if the entrants' observed ranges overlap, no winner is declared. At five iterations a 10% gap in means routinely sits inside a 3x spread

ModelModality.ts	167 lines. Vision and embedding are identified positively from Ollama's families array; text is the residual and is always marked inferred, because the absence of a vision family is not proof of anything
TaskCompatibility.ts	174 lines. Turns a declared task type into a constraint. Deliberately asymmetric - vision can do text, text cannot do vision, embedding neither. shapeOf('code_generation') states plainly that nothing here compiles or runs the output
ComparisonEngine.ts	101 lines. The scaffold's original comparison helper

Connected folders

- ../../application/services/ - the runner that calls these.
- ../../application/use-cases/ - RunComparison, which uses the planner and the report.
- ../../infrastructure/ - OllamaResidencyProbe.ts supplies the observation hardware fit scores.
- src/core/logging/ - the session log, which must not undo the egress warning.
- tests/benchmark/ - hardware-fit, privacy-assessor, egress-warning, comparison, task-compatibility.
- evidence/evaluation/ - the ten-case matrix drives these directly.

Referenced from

These folders point here. Each link below resolves in both directions.

- scripts/evaluation/
- src/app/api/v1/comparisons/
- src/app/api/v1/readiness/
- src/app/api/v1/readiness/[id]/ (FOLDER.md)
- src/modules/benchmark/core/
- src/modules/benchmark/core/entities/

src/modules/benchmark/infrastructure/providers/ - the adapters

Work package: benchmark and provider layer.

What this folder is

Nine files: one base class, four provider adapters, a registry, the error vocabulary, and two transport helpers.

The shape is deliberate. A subclass implements exactly one thing - how to stream one generation out of one provider (`streamOnce`). Timing, timeout enforcement, error classification, iteration and narrowing to the shared result type all happen in the base class, so **the four adapters cannot drift apart in how they measure or how they fail**.

What happened here

Unchanged in this pass, which is worth noting given how much moved around them. The adapters were already correct; what changed was what the layers above them were allowed to conclude from their output.

Two properties they already had are what made the rest possible. Every adapter takes its `fetch` and its clock by **injection**, so the whole suite runs with no network and no real timers. And **nothing here throws across the module boundary**: a failed iteration is still a measurement, and the reliability score is computed from how many iterations succeeded - so swallowing a failure would inflate it.

Files

File	What it is
BaseProvider.ts	328 lines. Timing, timeouts, error classification, iteration. <code>ProviderFailureError</code> and its brand
OllamaProvider.ts	167 lines. The local adapter. <code>POST /api/generate</code> with <code>stream: true</code> returns newline-delimited JSON, one object per token, with durations in nanoseconds . It streams even though the tokens are never displayed, because time-to-first-token cannot be measured from a buffered response - with <code>stream: false</code> the first byte and the last byte arrive together
GroqProvider.ts	176 lines. OpenAI-compatible SSE. Usage is omitted from streamed chunks unless <code>stream_options: { include_usage: true }</code> is sent. Groq also reports <code>x_groq.usage.completion_time</code> in seconds - server-side generation time excluding queueing, a better throughput denominator than our wall clock
GeminiProvider.ts	182 lines. <code>:streamGenerateContent</code> with <code>alt=sse</code> . <code>usageMetadata</code> repeats across chunks, so the last one wins. The key travels in the <code>x-goog-api-key</code> header, never the query string - a URL lands in server logs and proxy logs; a header does not
DemoProvider.ts	134 lines. The documented substitute the acceptance criteria allow when no real provider is reachable. It produces no measurements : every figure is labelled <code>simulated</code> , it is never registered unless <code>BENCHMARK_ALLOW_DEMO=true</code> , and the runner marks any run that touched it. Deterministic on purpose, so a demo is reproducible
ProviderRegistry.ts	152 lines. Which adapters exist, which are usable here, in what order to try them. Built from a <code>BenchmarkConfig</code> rather than reading <code>process.env</code> itself, so a test can construct any environment without mutating global state

errors.ts	199 lines. The published failure vocabulary. Every provider failure is mapped to one of these codes before it leaves an adapter. docs/local-model-setup.md lists them and the API returns their statuses
http.ts	151 lines. SSE and line-delimited stream readers, tokensPerSecond, the injected clock. A deliberate copy of the vision module's equivalent rather than a shared import, so the two modules can change independently
types.ts	129 lines. The measurement vocabulary: MeasurementProvenance, MeasurementStatus, ProviderUsage. Every number that leaves this module carries a label saying how it was obtained

Connected folders

- ../../core/ports/ - the AIProvider interface.
- ../ - config.ts builds the registry.
- src/app/api/v1/providers/ - the catalogue endpoint.
- tests/benchmark/ - one suite per adapter, plus provider-errors and provider-registry.
- docs/benchmark/ - the research log cites the official streaming contracts for all three.

src/modules/benchmark/infrastructure/repositories/ - persistence

Work package: benchmark and provider layer, over the integration layer's schema.

What this folder is

The only code in the benchmark module that talks to Prisma.

What happened here

`PrismaBenchmarkContext.ts` shrank from 46 lines to 35, because the device lookup went with the device table. `PrismaBenchmarkRepository.ts` gained the warmup flag on iteration rows and the nullable `hardwareFit`.

Three mapping rules hold here and are written into the files:

- The port speaks camelCase; the database is snake_case. The `@map()` directives in `schema.prisma` do that translation, so nothing here has to.
- `evidence` and `limitations` are Json columns, typed by Prisma as `JsonValue`, cast back to `string[]` on the way out. The entity declares `string[]` and the writer only ever puts strings in.
- **Nothing here swallows an error.** A caller that wants to degrade gracefully when the database is down decides that for itself - and `RunBenchmark` does exactly that, returning the measurement with `persisted: false`. A repository that silently reported success would be worse than one that fails loudly.

Files

File	What it is
<code>PrismaBenchmarkRepository.ts</code>	255 lines. The <code>BenchmarkRepository</code> implementation - saving runs, iterations and readiness scores, and reading them back
<code>PrismaBenchmarkContext.ts</code>	36 lines. The narrow lookup the use case needs: who owns the workload, and what uuid a provider slug maps to. Two small queries with <code>select</code> clauses rather than full row fetches - the prompt column on a workload can be large

Connected folders

- `../../core/ports/` - the interface implemented.
- `../../application/use-cases/` - the caller that decides how to degrade.
- `src/lib/` - the `PrismaClient` singleton.
- `prisma/` - the schema and its `@map()` directives.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/benchmark/core/entities/`
- `src/modules/benchmark/infrastructure/`